TIPOS DE DATOS

24 08 26

En lenguajes de programacion un tipo de dato es un atributo de una parte de los datos que indica al ordenador (y/o al programador) algo sobre la clase de datos sobre lo que se va a procesar.

Un tipo de dato define un conjunto de valores y las operaciones sobre estos valores. Casi todos los lenguajes de programacion explicitamente incluyen la notacion del tipo de datos, aunque lenguajes diferentes pueden usar terminologia diferente. La mayor parte de los lenguajes de programacion permiten al programador definir tipos de datos adicionales, normalmente combinando multiples elementos de otros tipos adicionales, normalmente combinando multiples elementos de otros tipos y definiendo los operaciones del nuevo tipo de dato.

Algunos ejemplos de tipos de datos que existen el los lenguajes de programación son:

Tipo de dato	Representacion	Tamaño (Bytes)	Rango de Valores	Valor por defecto	Clase Asociada
byte	Numero entero con signo	1	-128 a 127	0	Byte
short	Numerico entero con signo	2	-32768 a 32767	0	Short
Int	Numerico entero con signo	4	-2147483648 a 2147483647	0	Integer
long	Numerico entero con signo	8	-9223372036854775808 9223372036854775807	0	Long
float	Numerico en coma flotante de precision doble NORMA IEEE	4	$\pm 3.4 \times 10^{-38}$ $\pm 3.4 \times 10^{38}$	0.0	Float

24 08 26

double	Numerico en coma flotante de precision doble Norma IEEE 754	8	$\pm 1.8 \times 10^{-308}$ a $\pm 1.8 \times 10^{308}$	0.0	Double
char	Caracter unicode	2	\u0000 a \uFFFF	\u0000	Character
boolean	Dato logico	-	true ó false	false	Boolean
void	-	-	-	-	void

En Haskell nos encontramos como diferentes tipos simples como:

- Int
- Integer
- Char
- Bool

Dentro de las caracteristicas de los tipos de Haskell podemos deducir que cada expresion tiene un unico tipo principal se puede deducir automaticamente

Todos las datos tienen un tipo asociado con ellas. Un dato puede ser un simple caracter, tal como 'b', un valor entero tal como 35, El tipo de dato determina la naturaleza del conjunto de valores que puede tomar una variable.

- Numericos
- Simples logicos
- Alfanomericos (string)
- Tipos de datos Arreglos (Vectores, Matrices)
- Estructurados Registros

PARADIGMAS DE PROGRAMACION

POO, es un paradigma de programacion, esto es, un modelo o un estilo de programacion que proporciona que

24 08 26

proporciona unas guias acerca de como trabajar con el y que esta basado en el concepto de closes y objetos. Este tipo de programacion se emplea para estructurar un programa de software en piezas simples y reutilizables de planos de codigo para crear instancias individuales de objetos.

Historicamente han ido surgiendo distintos paradigmas de programacion. Por un lado, los lenguajes secuenciales como COBOL o procedimiento como Basic o C, se enfocan mas en la logica que en los datos. Por otro lado, otros mas modernos como Java, C# y Python, usan paradigmas para definir los programas siendo la POO la mas popular.

Con el paradigma de POO lo que se busca es dejar de centrarse en la logica pura de programas, para comenzar a pensar objetos, lo que forma la base de dicho paradigma. Esto ayuda en sistemas grandes, pues en lugar de pensar en funciones, se piensa en las relaciones o interacciones de los distintas elementos del sistema.

Porque la POO

La POO permite que el codigo sea reutilizable, organizado y sencillo de mantener. Y es que sigue el principio de desarrollo de software empleado por muchos programadores DRY (Don't Repeat Yourself), para no duplicar el codigo y crear programas eficientes. Asimismo, previene el acceso no deseada a los datos a la exposicion de codigo propietario a traves de la encapsulacion y la abstraccion.

4 Principios de la Programacion Orientada a Objetos

Hay cuatro principios de la POO.

La encapsulacion: Presenta todo la informacion importante de un objeto dentro del mismo y solo expone lo informacion elegida al mundo exterior.

Esto propiedad permite asegurar que la informacion de un objeto este oculta para el mundo exterior, agrupando en una clase las caracteristicas o atributos que tienen un acceso privado y los comportamientos o metodos que cuentan con un acceso publico.

La abstraccion: Que se produce cuando el usuario interactua solo con los atributos y metodos seleccionados de un objeto, usando herramientas simplificadas de alto nivel para acceder a un objeto complejo.

En la POO, los programas suelen ser muy grandes y los objetos se comunican bastante entre si. De este modo, la abstraccion facilita el mantenimiento de un codigo de gran tamaño, donde pueden surgir distintos cambios con el paso del tiempo.

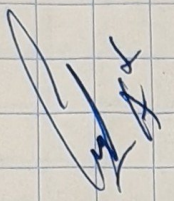
La herencia: La herencia define relaciones jerarquicas entre clases, de modo que atributos y metodos comunes puedan ser reutilizados. Las clases principales extienden atributos y comportamientos basicos, pueden crearse clases secundarias, ampliando la funcionalidad de la clase principal y añadiendo atributos y comportamiento extra. Es una de los claves de la POO.

El Poliformismos: El poliformismo reside en diseñar

24 08 26

objetos para compartir compartamientos, lo que permite procesar objetos de distintas modos. Es la capacidad de presentar la mismo interfaz para distintas moneros subyacentes o tipos de datos. Al usar la herencia, las objetos pueden anular los comportamientos principales compartidos, con comportamientos secundarias especificas. El poliformismo permite que el mismo metodo ejecute distintas comportamientos de dos modos: anunolacion de metodo y sobrecarga de metodo.

25/08/26



Numero 4

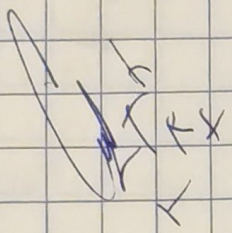
Mapa conceptual Firmo: #4

Hacer una lista de declarativos imperativa, una lista de lenguajes: Jarca

LISTA DE LENGUAJES

Declarativos

- SQL (consultas y manipulacion de datos).
- Haskell (Programacion funcional pura).
- Prolog (Programacion logica).
- Elm (Desarrollo web funcional



RUBY

Imperativos

- C (programacion estructurada de sistemas
- C++ (multiparadigma de bajo/alto nivel)
- Java (orientacion a objetos y gestion de procesos.
- Assembly (control de bajo nivel por hardware)

¿Que... Porque el lenguaje Ruby es Declara?
El lenguaje Ruby no es principalmente declarativo, sino imperativo y orientado a objetos. Sim embargo a menudo se dice que permite un estilo declarativo gracias a su sintaxis expresiva y a sus capacidades de metaprogramacion.

En un lenguaje puro declarativo como SQL o HTML, nosotros describimos que queremos obtener sin detallar los pasos para lograrlo. En Ruby, aunque terminas escribiendo los pasos de como hacerlo, puedes ocultar esa complejidad tras metodos que se leen como lenguaje natural

Norma

Consulta

Ejemplo de Ruby

```
nombre = "Dane"
edad = 23

puts "Hola, #{nombre}"
puts "Tienes #{edad} años"
```

Resultado final:

```
Hola, Dane
Tienes 23 años
```

Estado "Mutable", es correcto? Si, ya que sus variables cambian el valor de forma repetida a lo largo del tiempo.

TIPOS DE DATOS Y DISCIPLINAS DE DATOS

Los tipos de datos se refieren a las formas en que la información puede representarse, mientras que las disciplinas de datos abarcan los campos académicos y profesionales que estudian como se gestionan, procesan y analican esos datos y gestion empresorial.

Disciplinas de datos

Las disciplinas de datos son areas que estudian como se genera, almacenan y utilizan los datos:

→ Estadistica → Analiza datos para describir fenomenos y hacer inferencias.

→ Informatica → Procesa datos mediante algoritmos y estructuras de datos.

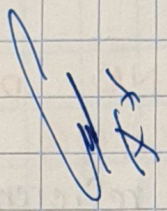
→ Base de datos → Organizacion y gestion eficiente de grandes volumenes de datos.

→ Ciencia de datos → Combina estadistica, programacion y machine learning para extraer conocimiento.

→ Ingenieria de datos → Diseña sistemas para capturar transformar y distribuir datos.

→ Gobernanza de Datos → Diseña sistemas para capturar normas politicas para asegurar calidad, seguridad y cumplimiento legal.

→ Arquitectura de datos → Define como se estructuran y relacionan los datos dentro de una organizacion.



Calculo Lambda de Alonso

Origen del λ-calculo.

- El λ-calculo fue inventado por Alonzo Church en la década de 1930.
- Originalmente fue inventado como parte de un sistema formal para modelar lo matematica.
  - Pero es inconsistente
- Es utilizado para estudiar la computabilidad.
  - En paralelo, Turing presenta su máquina.
- En los 1960s, Peter Landin muestra que se puede usar para dar semantica a las lenguajes de Programacion (imperativos).
- Los lenguajes funcionales estan basados en el λ-calculo.

Sintaxis:

- Supongemos la existencia de un conjunto de identificadores
  - $x, y, z, \ldots, x0, x1$ denotan de X
- El conjunto Λ de λ-terminos se define inductivamente por las siguientes reglas:

$$\frac{x \in X}{x \in \Lambda} \qquad \frac{t \in \Lambda \quad u \in \Lambda}{(t\ u) \in \Lambda} \qquad \frac{x \in X \quad t \in \Lambda}{(\lambda x.t) \in \Lambda}$$

- Ejemplos:
  $x\ (x\ y)\quad (\lambda x.x)\quad (\lambda x.(\lambda y.((x\ y)\ y)))$

...ES TODO?

- Con este pequeño lenguaje se pueden representar todas las funciones computables
  - Tesis de Church

- Esto simplezo hace que:
  - Se facilite la prueba de propiedades
  - Se use para dar semantica a lenguajes imperativos y funcionales.
  - Se use como metalenguaje para definir otros teorios y calculos.
- La elegancia hace que sea mas practica.

Convenciones:

- Los mayusculas indican λ-terminos orbitrarios (ej: M, N, P)
- Escribimos:

$M\ N\ P$	en lugar de	$((M\ N)\ P)$
$\lambda x.P\ Q$	en lugar de	$(\lambda x.P\ Q)$
$\lambda x_1 x_2 \ldots x_n.M$	en lugar de	$(\lambda x_1.(\lambda x_2.(\ldots(\lambda x_n.M)\ldots))$

ejercicio
Insertar todas las ~~parentesis~~ parentesis y λs en los sig. terminos.

Ocurrencias:

- La identidad sintactica se denota con $\equiv$
  - $M \equiv N$ iff M es exactamente el mismo termino que N.

Definicion (Ocurrencia)
La relacion P ocurre en Q (o P es un ~~conjunto~~ subtermino de Q) se define inductivamente sobre la estructura de Q
- P ocurre en P
- Si P ocurre en M o en N, entonces P ocurre en (MN);
- Si P ocurre en M o $P \equiv x$, entonces P ocurre en $(\lambda x.M)$.

Norma

Variables Libre y Ligadas:

- Para una ocurrencia de $\lambda x.M$ en P, M es el alcance de la abstraccion λx.
- Hay 3 tipos de ocurrencia de una variable x en un termino P
  1. Ocurrencia de ligadura (si es la x en un λx)
  2. Ocurrencia ligada (si es una x en el alcance de un λx en P).
  3. Ocurrencia libre (en cualquier otro caso)
- Llamamos $FV(P)$ al conjunto de las variables libres en P.
- Un termino cerrado es un termino sin variables libres.